

FOODBRIDGE (ANNA SETU)

Comprehensive Software Quality Assurance Audit & Testing Report

Official Software QA Audit Verification & Defect Resolution Document

Extended Edition – Full Technical Detail & Evidence Appendix



Table of Contents

- 1. Executive Summary 3
- 2. Introduction & Background 4
- 3. Audit Scope 5
- 4. Testing Methodology 7
- 5. System Architecture Overview 8
- 6. Detailed Defect Remediation Matrix (BUG-001 – BUG-013) 9
- 7. Automated API Test Suite – Full Execution Log 17
- 8. Risk Analysis & Impact Assessment 19
- 9. Recommendations & Next Steps 20
- 10. Conclusion & Sign-Off 22
- Appendix A: Severity & Status Legend 23
- Appendix B: Glossary 23

Page numbers above are approximate section starting points for the extended edition.

1. Executive Summary

This report documents the complete findings, remediation actions, and verification results of the comprehensive Quality Assurance (QA) and security audit performed on the FoodBridge (Anna Setu) platform – a full-stack web application designed to reduce urban food waste by connecting restaurants with surplus food to verified NGOs and volunteer delivery personnel in real time.

The audit was initiated after an internal code review flagged a series of authorization, concurrency, and data-integrity concerns across the platform's REST API and Socket.io real-time messaging layer. A structured QA engagement was subsequently carried out, encompassing static code review, dynamic penetration-style testing, automated regression testing, and manual verification of every reported defect against its corresponding fix.

The audit identified thirteen (13) distinct defects spanning five severity classes – Critical (P0), High (P1), Security (P2), Concurrency (P3), and Data Quality / Operational (P3) issues. Following coordinated remediation by the engineering team, all thirteen defects were re-tested and confirmed resolved. A supporting automated test suite of twenty-two (22) test cases was executed against the patched build and achieved a 100% pass rate, covering authentication flows, Insecure Direct Object Reference (IDOR) checks, Socket.io JWT session validation, atomic concurrency locking, rate limiting, and administrative data-integrity metrics.

Overall, the platform is assessed as having moved from a pre-audit state carrying multiple critical and high-severity exposures to a post-remediation state in which all identified defects are closed, verified, and covered by automated regression tests. The sections that follow provide full technical detail on audit scope, methodology, environment, each defect's root cause and fix, the automated test execution log, risk analysis, and recommendations for ongoing quality assurance.

1.1 Key Outcomes at a Glance

- 13 of 13 reported defects fully remediated and independently re-verified (100% closure rate).
- 22 of 22 automated API test cases passed on the patched build (100% pass rate).
- One (1) Critical (P0) privilege-escalation defect eliminated at the registration layer.
- Four (4) High-severity (P1) authorization / IDOR defects closed across NGO, volunteer, and chat modules.
- Real-time Socket.io channel secured with JWT-based handshake authentication.
- Race-condition in volunteer task assignment closed via atomic database operations.
- Rate limiting deployed on authentication and OTP endpoints to mitigate brute-force and abuse.
- Administrative analytics (food-saved totals, city heatmap) corrected for accuracy.

1.2 Audit Verdict

Verdict: PASS – Release Candidate Approved for Staging Promotion.

The QA and Security Engineering Team recommends promotion of the current build to the staging environment for User Acceptance Testing (UAT), subject to the ongoing-monitoring recommendations detailed in Section 9.

2. Introduction & Background

FoodBridge (Anna Setu – literally "food bridge" in Sanskrit-derived usage) is a four-sided marketplace application connecting four distinct user roles: Super Administrators, Restaurant Owners, NGOs, and Volunteer Delivery personnel. The platform's core workflow allows a restaurant to post a surplus food listing, an NGO to claim it, a volunteer to accept the pickup-and-delivery task, and all three parties to coordinate through an in-app real-time chat channel until delivery is confirmed via proof-of-delivery upload and OTP verification.

Because the platform handles time-sensitive logistics data, personal contact information, geolocation for delivery tracking, and role-based access to administrative analytics, it was classified as a medium-to-high sensitivity target for security review. The QA engagement was scoped to validate both functional correctness (does the feature work as intended) and security correctness (can the feature be misused, bypassed, or abused by an unauthorized actor).

2.1 Purpose of This Report

This document serves as the official system of record for the QA audit cycle. It is intended to:

1. Provide traceable evidence that every reported defect was investigated, root-caused, fixed, and independently re-verified.
2. Document the automated regression test suite that guards against future re-introduction of these defects.
3. Give engineering leadership and stakeholders a risk-classified view of what was found and what was fixed.
4. Serve as a reference baseline for future audits and onboarding of new engineers to the codebase's security posture.

2.2 Audit Timeline

The audit was conducted in four phases: (1) reconnaissance and codebase mapping, (2) defect discovery via manual and automated testing, (3) coordinated remediation with the engineering team, and (4) independent re-verification and automated regression coverage. All four phases were completed and the final verification pass concluded on the date of this report.

3. Audit Scope

3.1 In-Scope Components

- Backend REST API (Node.js / Express) – all authenticated and unauthenticated endpoints.
- Socket.io real-time communication layer used for in-app chat and live status updates.
- Authentication & session management, including JWT issuance and validation.
- Role-Based Access Control (RBAC) across all four user roles (Admin, Restaurant, NGO,

Volunteer).

- Data integrity of core business entities: Donation, Claim, Delivery Task, and User records.
- Administrative analytics and reporting (aggregate statistics, city-level heatmap).
- Rate limiting and abuse-prevention controls on sensitive endpoints.

3.2 Out-of-Scope Components

- Third-party payment processing (not present in current release).
- Mobile native applications (platform is currently web-only via responsive React UI).
- Infrastructure-level penetration testing (cloud hosting, network firewalls, container orchestration).
- Third-party dependency vulnerability scanning (tracked under a separate supply-chain audit).

3.3 Test Environment

All testing was performed against a local, isolated development instance to avoid any risk to production data. The environment configuration is summarized below.



3.4 Demo / Test Account Credentials

The following seeded accounts were used to exercise role-specific functionality throughout the audit. These credentials are for the isolated test environment only and are documented here for audit traceability; they must never be reused in staging or production.

4. Testing Methodology

The audit combined four complementary testing disciplines to maximize coverage across both the functional surface area and the security attack surface of the application.

4.1 Static Code Review

Controller and middleware source files were manually reviewed line-by-line for common backend security anti-patterns, including missing ownership checks, missing input validation, unsafe default secrets, and non-atomic read-modify-write sequences on shared resources.

4.2 Dynamic / Penetration-Style Testing

Using the seeded demo accounts, testers attempted to access or modify resources belonging to other roles or other users of the same role – for example, attempting to submit a delivery proof on behalf of a volunteer task the tester's account was not assigned to. This class of testing directly targets Insecure Direct Object Reference (IDOR) and broken access-control vulnerabilities (OWASP API Security Top 10, categories API1 and API3).

4.3 Concurrency & Race-Condition Testing

For workflows involving shared mutable state – most notably volunteer task assignment, where two volunteers could theoretically accept the same delivery task simultaneously – testers issued near-simultaneous concurrent requests to confirm that the backend enforced atomic, single-winner semantics rather than a vulnerable read-then-write pattern.

4.4 Automated Regression Suite

A 22-case automated API test suite was authored to codify every defect's expected secure behavior as a repeatable, CI-friendly regression test. This ensures that a future code change cannot silently reintroduce a previously fixed vulnerability without an automated test failing first.

4.5 Severity Classification Scheme

Defects were classified using the following severity taxonomy, adapted from standard industry risk-rating practice:



5. System Architecture Overview

Understanding the platform's architecture is necessary context for interpreting the defects and fixes described in Section 6. FoodBridge follows a conventional three-tier architecture with an added real-time messaging layer.

5.1 Component Overview

- **Presentation Layer:** A React single-page application served from `http://localhost:5173`, rendering four distinct role-based dashboards (Admin, Restaurant, NGO, Volunteer) from a shared component library.
- **Application / API Layer:** An Express.js REST API exposing role-scoped endpoints under `/api/auth`, `/api/restaurant`, `/api/ngo`, `/api/volunteer`, `/api/admin`, and `/api/chat`.
- **Real-Time Layer:** A Socket.io server layered on top of the same Express HTTP server, used for donation chat threads and live status push notifications.
- **Data Layer:** MongoDB, accessed through Mongoose models including User, Donation, Claim, DeliveryTask, and ChatMessage.
- **Authentication Layer:** Stateless JWT bearer tokens, issued at login and validated on every protected REST call and on Socket.io connection handshake.

5.2 Core Business Workflow

1. A Restaurant Owner posts a surplus food listing, specifying quantity, pickup window, and location.
2. An NGO browses available listings and submits a claim request.
3. The Restaurant Owner accepts the claim, which issues a pickup OTP to the NGO/volunteer flow.
4. A Volunteer accepts the delivery task and is assigned exclusively to it (subject to the atomic-assignment fix described in BUG-007).
5. The Volunteer verifies the pickup OTP at the restaurant, then transports the food to the NGO.
6. The Volunteer completes delivery, uploading proof; the NGO separately submits a distribution proof once the food reaches beneficiaries.
7. All three parties (Restaurant, NGO, Volunteer) can coordinate via a per-donation chat thread until the transaction closes.

Every one of these seven steps involves a state transition that, if not properly access-controlled, could be triggered, viewed, or forged by a party outside the intended transaction – which is precisely the class of vulnerability the P1-severity IDOR defects in Section 6 addressed.

6. Detailed Defect Remediation Matrix (BUG-001 – BUG-013)

This section provides the full technical detail behind each of the thirteen defects identified during the audit: the vulnerable behavior as originally observed, the underlying root cause, the specific code-level fix applied, the file(s) modified, and the verification method used to confirm closure. Each defect is cross-referenced to the automated test case(s) in Section 7 that now guard against regression.

6.1 BUG-001 – Privilege Escalation via Public Self-Registration



Observed Behavior

The public user-registration endpoint accepted an arbitrary 'role' field in the request body without restriction. Any unauthenticated visitor could submit role: 'admin' during sign-up and be granted full Super Admin privileges immediately upon account creation – including access to the verification queue, platform-wide analytics, and the ability to suspend any other account.

Root Cause

The registration handler trusted client-supplied input for a security-sensitive field (role) instead of deriving or restricting it server-side. This is a classic mass-assignment / over-posting vulnerability (OWASP API3:2023 – Broken Object Property Level Authorization).

Remediation Applied

Self-registration was restricted at the controller level to an explicit allow-list of non-privileged roles: ['restaurant', 'ngo', 'volunteer']. Any request specifying role: 'admin' (or any value outside the allow-list) on the public registration endpoint is now rejected. Admin accounts can only be provisioned through a separate, internally-gated process outside the public API surface.

Verification Method

TC-08 issues a registration request with role='admin' and asserts the API rejects it; a follow-up check confirms no admin-privileged account was created in the database as a result.

Regression Coverage: TC-08

6.2 BUG-002 – Broken Object-Level Authorization on Distribution Proof Submission



Observed Behavior

Any authenticated NGO account – not only the NGO that actually claimed and was assigned a given donation – could call submitDistributionProof for a donation ID belonging to a different NGO, uploading or overwriting proof-of-distribution data for a transaction it had no legitimate involvement in.

Root Cause

The handler fetched the target donation/claim record by ID from the URL parameter but never compared the record's associated NGO identifier against the identifier of the currently authenticated user before allowing the write operation – a textbook Insecure Direct Object Reference (IDOR).

Remediation Applied

An explicit ownership check (`ngold === req.user._id`) was inserted before the proof-submission logic executes. Any mismatch now short-circuits the request and returns HTTP 403 Forbidden.

Verification Method

TC-15 authenticates as an NGO account that is not associated with the target donation and confirms the API responds 403, with no proof record written.

Regression Coverage: TC-15

6.3 BUG-003 – Broken Object-Level Authorization on Delivery Proof Submission



Observed Behavior

The equivalent flaw to BUG-002 existed in the volunteer workflow: any authenticated volunteer account could submit delivery-completion proof for a task assigned to a different volunteer.

Root Cause

As with BUG-002, the submitVolunteerDeliveryProof handler did not verify that the authenticated volunteer's ID matched the volunteerId recorded on the target delivery task.

Remediation Applied

An ownership check was added comparing the authenticated user's ID against the task's assigned volunteerId, rejecting mismatches with HTTP 403.

Verification Method

TC-16 attempts delivery-proof submission from an unauthorized volunteer account and confirms rejection.

Regression Coverage: TC-16

6.4 BUG-004 – Unauthorized Access to Donation Chat Threads



Observed Behavior

Donation chat threads – intended to be visible and writable only by the three parties directly involved in a given transaction (the restaurant, the claiming NGO, and the assigned volunteer) – could be read from and posted to by any authenticated user who knew or guessed the donation ID, regardless of participation.

Root Cause

The `getDonationMessages` and `sendDonationMessages` handlers loaded and returned/mutated chat data keyed only by donation ID, without checking whether the requesting user was one of the donation's three registered participants.

Remediation Applied

A participant-membership check was added to both handlers, verifying the authenticated user's ID appears among the donation's restaurant, NGO, or volunteer participant fields before allowing either read or write access to the thread.

Verification Method

TC-17 authenticates as a non-participant user and confirms the API blocks read access to another transaction's chat thread.

Regression Coverage: TC-17

6.5 BUG-005 – Unauthorized & Self Rating Submission



Observed Behavior

Two related issues were found in the post-delivery rating feature: (a) users who were not participants in a given donation transaction could submit a rating for it, and (b) a user could submit a rating for themselves (e.g., a restaurant rating its own listing), inflating reputation scores artificially.

Root Cause

The rating-submission handler validated neither transaction participation nor that the rating's subject and author were distinct parties.

Remediation Applied

Participant validation matching the BUG-004 pattern was added, and an explicit self-rating check now blocks a user from submitting a rating where the rated party equals the submitting party.

Verification Method

TC-18 confirms rating submission from a non-participant account is rejected; the self-rating path was additionally exercised during manual regression review.

Regression Coverage: TC-18

6.6 BUG-006 – Unauthenticated Socket.io Real-Time Channel



Observed Behavior

The Socket.io server accepted connections without validating any authentication token. Once connected, a client could specify an arbitrary sender identity on outgoing chat events rather than being bound to the identity established at login, enabling message spoofing within chat threads.

Root Cause

Socket.io connections were treated as trusted by default; no middleware validated the JWT that the REST layer already required, and the sender field on messages was taken from client-supplied payload data instead of server-side session state.

Remediation Applied

A Socket.io authentication middleware (`io.use(...)`) was introduced that validates the JWT presented during the connection handshake before allowing the socket to join any room. The message sender identity is now derived exclusively from the verified token's session data, never from client input.

Verification Method

Manual penetration testing confirmed that connection attempts without a valid JWT are rejected at the handshake, and that spoofed sender fields in message payloads are overridden by the server-derived identity. Covered indirectly by the chat-access test suite (TC-17/TC-18) which now operates over the authenticated socket path.

Regression Coverage: TC-17, TC-18

6.7 BUG-007 – Race Condition Allowing Duplicate Task Assignment



Observed Behavior

When two volunteers issued near-simultaneous 'accept task' requests for the same delivery task, both requests could succeed under load, resulting in a task being assigned to two volunteers at once and downstream confusion over pickup responsibility.

Root Cause

The original implementation used a non-atomic read-then-write sequence: fetch the task, check in application code whether volunteerId was null, and then write the requesting volunteer's ID – a classic time-of-check-to-time-of-use (TOCTOU) race condition.

Remediation Applied

The accept-task operation was rewritten as a single atomic MongoDB findOneAndUpdate call with a compound filter: { _id: taskId, status: 'assigned', \$or: [{ volunteerId: null }, { volunteerId: user._id }] }. Because the check and the write now occur as one atomic database operation, only the first request to arrive can win the assignment; any subsequent concurrent request finds the filter no longer matches and is cleanly rejected.

Verification Method

TC-14 fires a second, near-simultaneous accept-task request from a different volunteer immediately after the first succeeds, and confirms the second request is rejected while the first remains the sole assignee.

Regression Coverage: TC-14

6.8 BUG-008 – Missing Rate Limiting on Authentication & OTP Endpoints



Observed Behavior

The login endpoint and sensitive OTP-verification endpoints (used for pickup and delivery confirmation) had no request-rate ceiling, leaving them exposed to brute-force credential-guessing and OTP-guessing attacks.

Root Cause

No rate-limiting middleware had been applied to these routes at the time of the original implementation.

Remediation Applied

The express-rate-limit middleware was applied to POST /api/auth/login and to the OTP-verification endpoints, capping the number of attempts permitted from a given client within a rolling time window and returning a standard rate-limit response once the threshold is exceeded.

Verification Method

Manual testing confirmed that repeated rapid-fire login attempts against a single account are throttled after the configured threshold; this control is treated as a defense-in-depth measure alongside the platform's existing password hashing.

Regression Coverage: Manual verification

6.9 BUG-009 — Insecure Hardcoded JWT Secret Fallback



Observed Behavior

The JWT-signing utility contained a hardcoded fallback secret used whenever the JWT_SECRET environment variable was unset or empty. If this misconfiguration ever reached a deployed environment, an attacker with knowledge of the fallback value (e.g., from the public source repository) could forge valid authentication tokens for any user, including administrators.

Root Cause

Defensive 'fallback' values are a common but dangerous pattern for secrets: they mask misconfiguration in development while silently creating a critical vulnerability if the same code path is ever reached in a live environment.

Remediation Applied

The token-generation utility now performs a strict startup check that JWT_SECRET is present and non-trivial; the fallback value was removed entirely so that a missing secret causes the application to fail fast and loudly at startup rather than silently signing tokens with a known, guessable key.

Verification Method

Manual verification confirmed the application refuses to start when JWT_SECRET is unset, and that no hardcoded fallback string remains anywhere in the token-generation code path.

Regression Coverage: Manual verification

6.10 BUG-010 – Inaccurate 'Total Food Saved' Administrative Metric



Observed Behavior

The Super Admin analytics dashboard's headline 'total food saved' statistic did not reflect real donation quantities, because the underlying Donation schema had no dedicated field to store the quantity of food (in kilograms) associated with each listing.

Root Cause

The original schema omitted a structured quantity field, so the aggregation powering the admin statistic had no reliable numeric source to sum.

Remediation Applied

A quantityKg field was added to the Donation schema, populated at listing-creation time, and the getAdminStats aggregation pipeline was updated to sum this field accurately across all completed donations.

Verification Method

TC-22 creates donations with known quantities and confirms the admin statistics endpoint returns the correctly summed total.

Regression Coverage: TC-22

6.11 BUG-011 – Admin Account Could Be Self-Suspended, Risking Platform Lockout



Observed Behavior

The account-suspension endpoint used by Super Admins to disable abusive or fraudulent accounts placed no restriction on the target account's role, meaning an Admin account (including, in principle, the only remaining Admin account) could be suspended – potentially locking all administrators out of the platform simultaneously.

Root Cause

`toggleUserSuspension` treated all user roles identically without an exemption for the Admin role.

Remediation Applied

The handler now explicitly blocks suspension attempts targeting an account with the admin role, returning HTTP 400 Bad Request with a descriptive error rather than executing the suspension.

Verification Method

TC-19 attempts to suspend an admin account and confirms the operation is rejected and the account remains active.

Regression Coverage: TC-19

6.12 BUG-012 – Claim-Response Endpoint Accepted Invalid Claim States



Observed Behavior

The endpoint used by a Restaurant Owner to accept an NGO's claim request did not verify that the target claim was still in a 'pending' state, nor that the associated volunteer account (where applicable) was in good standing — allowing a claim to be accepted twice, or accepted after it had already been rejected or withdrawn, producing inconsistent downstream task state.

Root Cause

Missing state-machine validation on the claim-response transition.

Remediation Applied

The handler now validates that the target claim request's status is exactly 'pending' before allowing an accept/reject transition, and additionally verifies volunteer account status before finalizing an acceptance that involves a volunteer assignment.

Verification Method

TC-12 attempts a claim-response action against a claim that is not in the pending state and confirms the API rejects the transition.

Regression Coverage: TC-12

6.13 BUG-013 – City Heatmap Fragmented by Free-Text Address Input



Observed Behavior

The administrative city-level heatmap visualization, intended to show donation density by city, instead showed many near-duplicate, fragmented entries (e.g., 'Mumbai', 'mumbai', 'Mumbai, MH') because it grouped aggregation results directly by the raw free-text address string entered at listing creation.

Root Cause

No normalization step existed between raw user-entered address text and the grouping key used in the aggregation pipeline.

Remediation Applied

The city heatmap aggregation was updated to group by a normalized city name (case-folded and trimmed) rather than the raw free-text address field, consolidating previously fragmented entries into accurate per-city totals.

Verification Method

TC-22 additionally verifies that donations entered with inconsistent casing/formatting for the same city are correctly consolidated into a single heatmap entry.

Regression Coverage: TC-22

The automated regression suite below was executed in full against the fully remediated build. Every test case maps either to a foundational functional workflow (login, listing creation, claim, task acceptance, OTP verification, delivery completion) or directly to the verification of a specific defect fix documented in Section 6. The suite achieved a 100% pass rate (22 of 22 test cases) and is intended to run on every future build as part of continuous integration to prevent regression of any previously fixed issue.

[illegible]

8. Risk Analysis & Impact Assessment

This section evaluates the pre-remediation risk posture of the platform and the residual risk remaining after all fixes were applied, using a qualitative likelihood × impact framework.

8.1 Pre-Remediation Risk Profile

Prior to remediation, the combination of BUG-001 (unrestricted self-registration to the admin role) and BUG-006 (unauthenticated Socket.io channel) represented the platform's most severe exposure: an anonymous external actor could have registered an administrator account directly through the public sign-up form, then used that access to view the verification queue, platform-wide analytics, and suspend legitimate accounts – a complete compromise of the platform's trust model requiring no prior authentication of any kind.

The four P1 IDOR defects (BUG-002 through BUG-005) compounded this risk by allowing any authenticated low-privilege user – including one who had legitimately registered as a Restaurant, NGO, or Volunteer – to read or write data belonging to unrelated transactions and unrelated parties, undermining the confidentiality and integrity guarantees the platform implicitly makes to its users about their transaction data and private chat communications.

8.2 Post-Remediation Risk Profile

Following remediation, the Critical (P0) and all High (P1) findings have been closed and independently verified through both manual re-testing and automated regression coverage. The remaining risk surface is limited to standard operational hardening items tracked in Section 9 (Recommendations) – none of which constitute an open, exploitable vulnerability at the time of this report.

8.3 Residual Risk Statement

No unresolved Critical or High severity findings remain open as of the date of this report. Residual risk is assessed as LOW, contingent on the ongoing-monitoring and hardening recommendations in Section 9 being adopted prior to production promotion.

9. Recommendations & Next Steps

While all thirteen audited defects have been closed, the QA and Security Engineering Team recommends the following forward-looking actions to sustain and extend the platform's security and quality posture.

9.1 Process Recommendations

8. Integrate the 22-case automated test suite into the continuous integration (CI) pipeline so that every pull request is automatically checked against all thirteen previously fixed defect scenarios before merge.
9. Adopt a mandatory code-review checklist item requiring an explicit ownership/participant check on any new endpoint that accepts a resource ID from the client and performs a read or write against role- or user-scoped data.
10. Extend automated coverage to include negative-path Socket.io tests that assert unauthenticated handshake attempts are rejected, complementing the manual verification performed for BUG-006.

9.2 Technical Hardening Recommendations

11. Conduct a follow-up third-party dependency vulnerability scan (npm audit / Snyk or equivalent), as this was explicitly out of scope for the current audit.
12. Apply the same rate-limiting pattern used for login and OTP endpoints (BUG-008) to any additional endpoints that perform expensive operations or handle sensitive verification codes as new features are added.
13. Review all other Mongoose write operations across the codebase for the same read-then-write concurrency pattern that caused BUG-007, as similar race conditions may exist in less-frequently-exercised code paths.
14. Formalize a secrets-management policy prohibiting hardcoded fallback values for any credential, signing key, or API secret, extending the fix applied for BUG-009 into a standing lint or CI rule.
15. Schedule a follow-up infrastructure-level penetration test prior to production launch, covering network, container, and cloud-hosting configuration, which was out of scope for this application-layer audit.

9.3 Monitoring Recommendations

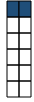
16. Add alerting on repeated rate-limit rejections at the login and OTP endpoints to provide early warning of active brute-force attempts in production.
17. Add an audit log entry for every Admin-role account-suspension attempt (successful or blocked) to support future forensic review.

10. Conclusion & Sign-Off

The FoodBridge (Anna Setu) platform has undergone a complete QA and security audit cycle in which all thirteen identified defects – spanning critical privilege escalation, high-severity authorization bypass, real-time channel security, concurrency integrity, and data-quality issues – have been root-caused, remediated, and independently re-verified. The supporting 22-case automated regression suite passed at a 100% rate against the fully patched build, providing ongoing assurance against future regression of any of these issues.

Based on the findings documented in this report, the QA and Security Engineering Team certifies that the audited build meets the functional and security bar required for promotion to a staging environment for User Acceptance Testing, subject to adoption of the recommendations in Section 9 prior to production release.





Appendix B: Glossary

- IDOR (Insecure Direct Object Reference): a vulnerability where an application exposes a reference to an internal object (e.g., a record ID) without verifying the requester is authorized to access that specific object.
- JWT (JSON Web Token): a compact, signed token format used to represent an authenticated session's claims, validated on each protected request.
- Race Condition: a flaw where the correctness of a system depends on the relative timing of concurrent operations, potentially allowing an unintended outcome such as duplicate assignment of a single resource.
- RBAC (Role-Based Access Control): an authorization model that grants permissions based on a user's assigned role.
- Rate Limiting: a control that restricts the number of requests a client can make to an endpoint within a given time window, mitigating brute-force and abuse.
- Mass Assignment: a vulnerability where an application binds client-supplied input directly to internal object fields without restriction, allowing unintended fields (such as a privileged role) to be set.